CREAR UNA API REST CON PHP, MVC Y JSON

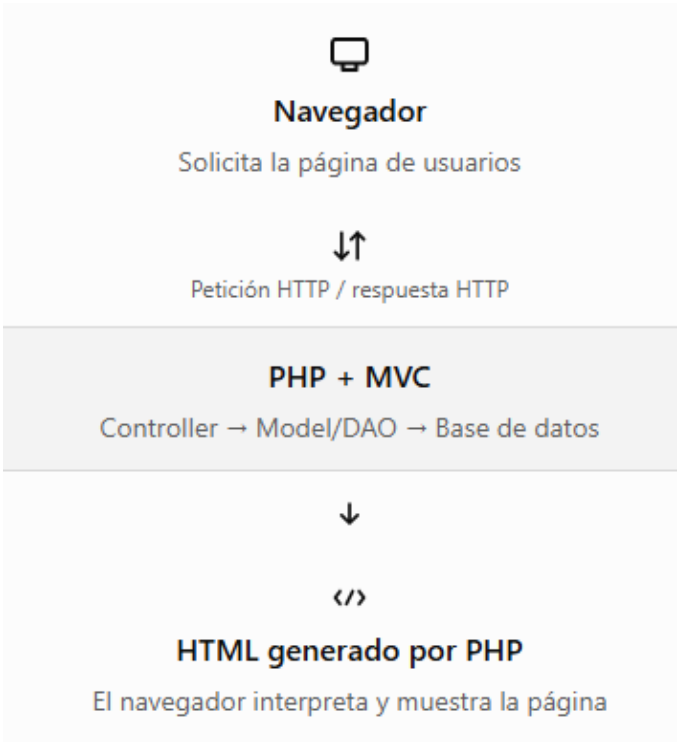
1. INTRODUCCIÓN

Una API permite reutilizar la lógica de una aplicación PHP para que otros programas puedan acceder a sus datos, sin necesidad de generar una página HTML.

2. ¿QUÉ ES UNA API Y PARA QUÉ SIRVE?

Una **API** (Application Programming Interface) es una interfaz que permite que dos aplicaciones se comuniquen entre sí mediante unas reglas definidas.

Imagina que tenemos una aplicación web que permite gestionar usuarios. Actualmente, el usuario entra en una página y el servidor genera HTML para mostrar la información.



Esto funciona perfectamente para una aplicación web tradicional. Sin embargo, ¿qué sucede si queremos que los mismos usuarios se puedan consultar desde una aplicación Android, otra página web hecha con JavaScript o un programa de escritorio?

No queremos crear una página HTML para cada aplicación. Queremos proporcionar los datos en un formato que todas ellas puedan interpretar.

Aquí es donde entra la API.



La diferencia fundamental es que una aplicación tradicional suele devolver HTML, mientras que una API de datos suele devolver JSON. Ambas pueden utilizar PHP, MVC, SQL y la misma base de datos.

3. ¿QUÉ ES JSON?

JSON (JavaScript Object Notation) es un formato de texto utilizado para representar e intercambiar datos entre aplicaciones.

Es independiente de PHP, Java, Kotlin o JavaScript: todos estos lenguajes pueden trabajar con JSON.

Por ejemplo, imaginemos que tenemos un usuario con los siguientes datos:

- ID: 1
- Nombre: Ane
- Apellidos: García
- Correo electrónico: ane@gmail.com

En PHP podríamos representar esos datos mediante un array asociativo:

```
$usuario = [
        "id" => 1,
        "nombre" => "Ane",
        "apellidos" => "García",
        "email" => "ane@ejemplo.com"
];
```

Pero si queremos enviarlos a otra aplicación, podemos convertirlos a JSON.

```
{
        "id": 1,
        "nombre": "Ane",
        "apellidos": "García",
        "email": "ane@ejemplo.com"
}
```

Observa que los datos son los mismos. Lo que cambia es la representación utilizada para intercambiarlos.

Reglas básicas de JSON

- **Objeto:** { "nombre": "Ane" }
- **Array:** [1, 2, 3]
- **Cadena de texto:** "nombre": "Ane"
- **Número:** "id": 1
- **Booleano:** "activo": true
- **Valor nulo:** "telefono": null

En JSON, las cadenas y los nombres de las propiedades utilizan comillas dobles. No se permiten comentarios ni comas sobrantes al final de los objetos o arrays.

¿Cómo convertimos datos PHP a JSON?

PHP dispone de la función **json_encode().**

```php
<?php
    $usuario = [
        "id" => 1,
        "nombre" => "Ane",
        "apellidos" => "García"
    ];
    $json = json_encode($usuario);
    echo $json;
?>
```

La respuesta será:

```
{"id":1,"nombre":"Ane","apellidos":"García"}
```

También podemos convertir el resultado de una consulta a la base de datos en JSON. Es decir, no tenemos que escribir los datos manualmente: podemos reutilizar los resultados del CRUD que ya sabemos implementar.

```php
$json = '{"nombre":"Ane","edad":20}';
$datos = json_decode($json, true);
echo $datos["nombre"]; // Ane
```

El segundo parámetro, true, hace que los objetos JSON se conviertan en arrays asociativos PHP.

4. ¿QUÉ ES UNA API REST?

REST es un estilo de diseño para construir servicios web que permiten acceder a recursos mediante HTTP.

Un recurso puede ser un usuario, un producto, un libro, una ubicación o una notificación.

Por ejemplo, si nuestra aplicación gestiona usuarios, podemos definir una API que permita realizar estas operaciones:

Consultar todos los usuarios: GET

/api/usuarios

Devuelve una lista

Consultar un usuario: GET

/api/usuarios/1

Devuelve un usuario

Crear un usuario: POST

/api/usuarios

Crea un usuario

Modificar un usuario: PUT

/api/usuarios/1

Actualiza un usuario

Eliminar un usuario: DELETE

/api/usuarios/1

Elimina un usuario

Estos métodos se corresponden con las operaciones que ya se conocen del CRUD.



Un aspecto importante: el método HTTP indica qué operación se quiere realizar. No es necesario crear una página HTML diferente para cada operación.

5. ¿CÓMO ENCAJA UNA API EN MVC?

Esta es probablemente la parte **más importante**: crear una API no significa abandonar MVC ni duplicar toda la aplicación.

Si ya tienen una aplicación estructurada en capas, pueden reutilizar gran parte del código.

Por ejemplo, si ya tienen:

- **Usuario.php:** representa los datos del usuario.
- **UsuarioDAO.php:** consulta, inserta, modifica y elimina usuarios en la base de datos.
- **UsuarioController.php:** coordina las operaciones.
- Una vista que genera una tabla HTML.

Pueden mantener el **modelo** y el **DAO**. Lo que cambia es la forma de presentar el resultado, es decir, el **Controller** y la **View** o **respuesta**.

En una aplicación MVC tradicional, el controlador suele enviar los datos a una vista PHP. En una API, el controlador puede devolver directamente una respuesta JSON, sin cargar una plantilla HTML.

Esto no obliga a que toda API tenga exactamente esta estructura: depende de cómo esté organizada la aplicación. Lo importante es conservar las responsabilidades separadas.